

LUMA: Environment-Conditioned Episodic Memory for Robust Code Agents under Software Drift

Ayush Raj

*School of Computer Engineering
KIIT University
Bhubaneswar, India
2305931@kiit.ac.in*

Hari Om Singh

*School of Computer Engineering
KIIT University
Bhubaneswar, India
2305936@kiit.ac.in*

Aarohan Garg

*School of Computer Engineering
KIIT University
Bhubaneswar, India
2305346@kiit.ac.in*

Akhand Pratap Singh Sikarwar

*School of Computer Engineering
KIIT University
Bhubaneswar, India
2305512@kiit.ac.in*

Akshita Arya

*School of Computer Engineering
KIIT University
Bhubaneswar, India
2305513@kiit.ac.in*

Anupam Sinha Mahapatra

*School of Computer Engineering
KIIT University
Bhubaneswar, India
23052867@kiit.ac.in*

Dr. Sujoy Dutta

*School of Computer Engineering
KIIT University
Bhubaneswar, India
sdattafcs@kiit.ac.in*

Abstract—Large language models have made it possible to build agents that can debug code, reason about software behavior, and generate full application stacks with less human involvement. Most of the multi-agent frameworks and RAG system, are built on an assumption that rarely gets stated: that retrieved knowledge stays valid over time, but it often does not. Libraries are regularly updated. APIs are frequently deprecated, system dependencies shift. A solution that worked against one environment may silently not work in another.

We tried to formalize this as Software Drift - the condition where a semantically plausible but version-incompatible solution from memory is retrieved by the agent and applies it to an environment that has since changed. The failures that comes next are not random; they recur precisely because the agent keeps trusting the same old retrieval. We call this pattern Negative Transfer. We propose LUMA (Large Unified Multi-Agent), a distributed orchestration engine built around a novel Tier-2 cognitive architecture called SOMA (Self Organising Memory architecture). SOMA models agent knowledge as a function conditioned on the current software environment, tagging each execution episode with a multi-dimensional environment hash and applying an Interference-Aware Retrieval algorithm that penalizes stale memories when drift is detected. We evaluate LUMA on a Software Drift Benchmark Suite spanning five API migration scenarios ($N=12,500$ controlled trials, $N=500$ grounding trials, $N=30$ real-world trials). Our results show that LUMA reduces stale retrieval error rates from 17.1% (Naïve RAG) to 8.0% across diverse drift conditions.

Index Terms—Multi-Agent Systems, Software Drift, Negative Transfer, Retrieval-Augmented Generation, Episodic Memory, Environment-Aware Memory, Autonomous Software Engineering, Large Language Models

I. INTRODUCTION

Large language models are rapidly changing how software gets built and maintained. Multi-agent systems composed of specialized LLM-driven roles, such as architects, coders, and critics, have shown strong results on complex programming tasks that benefit from collaborative problem solving [1], [2]. For these orchestration frameworks to work well over extended periods, they are increasingly paired with episodic or semantic memory modules, usually built on dense vector stores and Retrieval-Augmented Generation (RAG) [12]. This type of memory lets the agents recall previously successful strategies, reducing on unnecessary API usage, limiting hallucinations, and making reappearing bug-fixes faster to resolve.

Semantic retrieval assumes that similarity is equal to correctness. That assumption stands in constant domains. In software engineering, this tends to break quietly and frequently.

A standard RAG pipeline retrieves whichever past solution is closest to the current query and treats it as valid. It has no mechanism for verifying and making sure whether that solution still applies. `DataFrame.append()` fixed an `ImportError` in pandas v1.x - fine. Pandas v2.0 removes `append()`. The agent retrieves the same fix, try it, and crashes. Then retake it again, then crashes again. Its memory keeps ranking the broken fix highly, so the loop continues. We label this negative transfer.

Teams running AI coding assistants for production repositories will see this. Software dependencies do not sit still, most update on a cycle of weeks to months - while the agent's

memory accumulates without any awareness of those shifts. The mismatch grows over time.

A. Problem Formulation

Current memory-augmented agent systems coins strategy fetching as:

$$S = f(T, E) \quad (1)$$

where S is the retrieved strategy, E is the error context, and T is the task query. This formulation implicitly assumes that a valid strategy at time t_1 remains valid at any later time t_2 , regardless of environmental changes. We redefine the retrieval process as:

$$S = f(T, E, V) \quad (2)$$

where V captures the current system state: operating system, runtime version, and installed library versions. The objective is to minimize the *Negative Transfer Rate* (NTR) under drift while keeping the *Catastrophic Failure Rate* (CFR) low.

B. Research Questions

We investigate three research questions:

- **RQ1:** Does environment-conditioned retrieval reduce negative transfer compared to semantic-only retrieval under software drift?
- **RQ2:** Can active grounding preserve backward-compatible knowledge without introducing false positives?
- **RQ3:** Does the SOMA architecture maintain its reliability gains across varying agent capability levels?

C. Contributions

We present LUMA, a distributed orchestration framework built around SOMA. Our contributions are:

- 1) We formalize the Software Drift problem in LLM-based code agents and define Negative Transfer as a measurable failure mode.
- 2) SOMA, a novel environment-conditioned memory architecture that tags each stored strategy with a deterministic environment hash.
- 3) An Interference-Aware Retrieval mechanism that penalizes strategies validated under environments different from the current execution state.
- 4) An Active Grounding protocol that tests potentially outdated strategies in sandboxed environments before allowing reuse.
- 5) A Software Drift Benchmark Suite with large-scale experiments ($N=12,500$ controlled, $N=500$ grounding, $N=30$ real-world trials).

II. RELATED WORK

A. Autonomous Code Agents and Memory Architectures

Frameworks such as AutoGPT [1], MetaGPT [2], and SWE-agent [5] have demonstrated that autonomous agents can resolve issues and generate code within large software repositories, including challenging benchmarks like SWE-bench [6]. ReAct [21] showed how interleaving reasoning traces with

tool-use actions improves grounded decision-making, while AutoGen [23] introduced a flexible multi-agent conversation framework. ToolLLM [20] and CodeAgent [22] extended these ideas to tool-integrated code generation.

Several studies have proposed memory architectures for LLM agents. MemGPT [7] introduces a tiered storage hierarchy inspired by OS virtual memory, but its retrieval logic does not consider the software execution environment. Voyager [8] builds an expandable skill library for open-ended continual learning, yet assumes that validated skills remain correct indefinitely. Reflexion [9] uses verbal reinforcement with episodic memories but cannot distinguish failures caused by faulty reasoning from those caused by API deprecations.

The shared blind spot across all these systems is that their retrieval mechanisms have no mechanism for detecting whether the software environment has changed since a strategy was stored. SOMA fills this gap by separating semantic relevance from environment-aware validity.

B. Continual Learning and Catastrophic Forgetting

Catastrophic Forgetting has been studied extensively in the Continual Learning literature [10]. Existing dispersion - parameter isolation, experience replay, and regularization methods like Elastic Weight Consolidation (EWC) [11] - labels continuous statistical accumulations: gradual domain shifts, sensor noise distributions changing over time [13]. They assume change is smooth.

Software does not change polished. An API function is removed, a method signature can changes between two version numbers. A default parameter flips. These are discrete, structural events with no gradient to follow, the kind of change that existing continual learning methods were not built to handle. These are not gradual distribution shifts; they are hard breaks that cause deterministic failures. SOMA handles this by treating the software environment as a discrete gating variable rather than a continuously shifting distribution.

C. Automated Program Repair

Automated Program Repair (APR) aims to generate patches for faulty programs automatically [14], [15]. Traditional APR relies on search-based optimization and test-suite validation [14]. Recent neural approaches use historical bug-fix datasets and code language models [16]. APR systems, however, work at the level of localized syntactic corrections and do not build persistent memory of generalizable debugging strategies. LUMA goes further by converting debugging experiences into reusable semantic rules, each tagged with the software environment in which it was validated.

D. Research Gap

Table I makes the gap visible. None of the existing systems model environment-conditioned validity of stored strategies. To our knowledge, LUMA is the first framework that conditions memory retrieval on the active software environment state and includes an active grounding step to validate stored knowledge before reuse.

TABLE I
COMPARISON OF EXISTING SYSTEMS AND LUMA

System	Memory Type	Env. Aware	Drift Handling	Active Valid.
AutoGPT [1]	None	No	No	No
MemGPT [7]	Hierarchical	No	No	No
Voyager [8]	Skill Library	No	No	No
Reflexion [9]	Episodic	No	No	No
SWE-agent [5]	Exec.-based	No	No	No
Naïve RAG	Vector-based	No	No	No
LUMA (SOMA)	Tier-2 Cog.	Yes	Yes	Yes

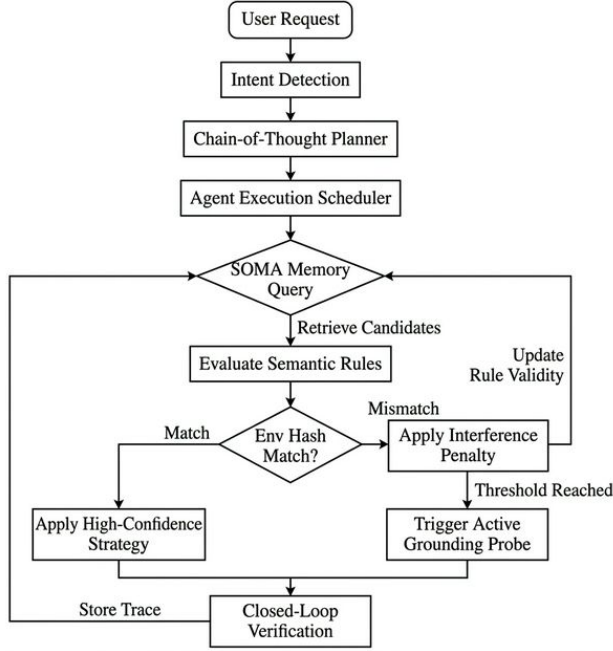


Fig. 1. The LUMA Framework Workflow, showing the end-to-end pipeline from user request through SOMA memory retrieval, environment hash validation, active grounding, and closed-loop verification.

III. METHODOLOGY

A. System Overview and Pipeline

LUMA is structured as a hierarchical multi-agent framework. At the top level, an orchestrator parses incoming user prompts through intent detection and uses Chain-of-Thought (CoT) reasoning to decompose complex requests into manageable subtasks. These subtasks are dispatched to specialized agents through an asynchronous task scheduler. The critical part of this execution process is the knowledge retrieval step: before any agent begins writing code or calling an API, it queries the SOMA subsystem to obtain strategic priors from past successful resolutions.

The end-to-end pipeline has six stages:

- 1) **Input Processing:** The user query is parsed into task description T and error context E .

- 2) **Task Decomposition:** The orchestrator splits tasks into subtasks and assigns them to specialized agents (planner, coder, executor, validator).
- 3) **Memory Retrieval:** Each agent queries SOMA with the current task context and environment state V . SOMA returns the best-scoring strategy after applying semantic similarity and the interference penalty.
- 4) **Code Execution:** The agent applies the saved strategy or, if none of the stored strategies is suitable, it generates a solution from scratch.
- 5) **Validation:** The output runs in a sandboxed environment to check for runtime errors and confirm that existing tests still pass.
- 6) **Memory Update:** Execution traces that passes are reserved as episodic entries. Over time, recurring patterns across these proofs are consolidated into semantic rules.

B. Module Design

The framework consists of five primary modules coordinated through a message-passing protocol:

- 1) **Orchestrator Module:** Manages task decomposition, agent coordination, and execution scheduling.
- 2) **SOMA Memory Module:** Stores episodic traces and consolidated semantic rules alongside an environment hash registry.
- 3) **Retrieval Engine:** Computes retrieval scores by combining semantic similarity with interference penalties.
- 4) **Grounding Module:** Runs the Active Grounding protocol and schedules sandboxed probes for unvalidated rules.
- 5) **Execution Agent Pool:** Four specialized agents (planner, coder, executor, validator) that carry out software engineering tasks and report outcomes back to the memory module.

C. Environment-Conditioned Memory Validity

SOMA differs from standard RAG implementations that employ metadata filtering (e.g., `filter: version == 2.0`). That approach assumes the orchestrator already knows which metadata fields matter at retrieval time. In practice, autonomous agents rarely have this prior information. An agent cannot predict that `pandas` deprecated `DataFrame.append()` in version 2.0 unless it has encountered a resulting failure or has explicit documentation stating the change.

SOMA dynamically maintains validity sets through probabilistic interference scoring and empirical sandbox probing. The architecture follows a Tier-2 cognitive design with two memory layers:

- **Hippocampal Layer (Episodic Memory):** Stores the raw log of execution traces, recording task description, the code patch attempted, the specific stack trace produced, and the success outcome.
- **Neocortical Layer (Semantic Memory):** Holds consolidated and abstracted strategies. During localized *sleep phases* (triggered at regular intervals), SOMA aggregates

episodic traces sharing similar error signatures into generalized Semantic Rules. The validated rules are explicitly persisted to the semantic store.

The environment hash is a deterministic fingerprint of the execution environment:

$$E_{\text{hash}} = \text{SHA256}(\text{OS} \parallel \text{Py_ver} \parallel \text{sorted}(D)) \quad (3)$$

where D is the set of installed packages and their pinned versions, captured via `pip freeze`.

Memory retrieval is formulated as a constrained optimization problem. Let q be the query task, d a stored semantic rule, and v the current environment state. The retrieval score is:

$$\text{Score}(q, d, v) = \text{sim}_{\text{sem}}(q, d) - \alpha \cdot \Gamma(v, d.\text{env_set}) \quad (4)$$

where $\text{sim}_{\text{sem}}(q, d)$ is the cosine similarity between query and rule embeddings computed via Sentence-BERT [18], $d.\text{env_set}$ is the set of environment hashes under which rule d has been empirically validated, and Γ is the interference penalty function:

$$\Gamma(v, S) = \begin{cases} 0 & \text{if } v \in S \\ 1 - \frac{|\text{overlap}(v, S)|}{|S|} & \text{otherwise} \end{cases} \quad (5)$$

The overlap function counts shared components (OS, Python version, individual libraries) between v and the nearest hash in S . A rule validated on the exact current environment carries zero penalty. A rule validated on a completely different environment incurs a penalty close to 1.0. We set $\alpha = 0.6$ based on a grid search over $[0.2, 1.0]$: values below 0.4 admitted too many stale retrievals, while values above 0.8 discarded useful backward-compatible rules.

D. Active Grounding and Curiosity Probing

Simply penalizing unfamiliar environments is insufficient: a rule might still work in the new environment. Discarding it would constitute catastrophic forgetting at the knowledge level.

SOMA addresses this with *Active Grounding*. When a stored rule scores high on semantic similarity but incurs a large interference penalty ($\Gamma > \tau_{\text{probe}}$), the rule is marked as stale. Instead of removing it, the LUMA orchestrator starts a curiosity-driven subroutine that schedules the rule for a probe evaluation (Algorithm 1). The execution agent applies the strategy inside a sandboxed Docker container. If the probe passes, the environment hash is added to the rule’s validity set and its penalty resets to zero. If it fails, the rule is blacklisted for that environment state - it will not be selected again under the same conditions.

In all experiments, $\tau_{\text{probe}} = 0.7$, $\alpha = 0.6$, and $\beta = 1.0$.

IV. IMPLEMENTATION DETAILS

A. System Stack

LUMA is implemented in Python 3.11. Table II. lists the full technology stack.

Algorithm 1 Active Grounding Probe

Require: q_t : query task, v : current env., $\mathcal{R}$: semantic rules
Ensure: d^* : best rule

```

1:  $\mathcal{C} \leftarrow \emptyset$ 
2: for each  $d \in \mathcal{R}$  do
3:    $s \leftarrow \text{cosine}(q_t, d.\text{content})$ 
4:   if  $v \notin d.\text{env\_set}$  then
5:     if  $s > \tau_{\text{probe}}$  then
6:        $\text{ok} \leftarrow \text{sandbox}(d.\text{strategy}, v)$ 
7:       if  $\text{ok}$  then
8:          $d.\text{env\_set} \leftarrow d.\text{env\_set} \cup \{v\}$  {forward-compatible}
9:          $p \leftarrow 0$ 
10:      else
11:         $d.\text{blacklist} \leftarrow d.\text{blacklist} \cup \{v\}$ 
12:         $p \leftarrow \beta$ 
13:      end if
14:    else
15:       $p \leftarrow \beta$ 
16:    end if
17:  else
18:     $p \leftarrow 0$ 
19:  end if
20:   $\mathcal{C} \leftarrow \mathcal{C} \cup \{(s - \alpha p, d)\}$ 
21: end for
22: return  $\arg \max_{(f, d) \in \mathcal{C}} f$ 

```

TABLE II
LUMA TECHNOLOGY STACK

Component	Technology
Orchestration	Custom Python controller with <code>asyncio</code>
Vector Storage	FAISS [17]
Embedding Model	all-MiniLM-L6-v2 [18]
Sandbox Execution	Docker containers [19]
LLM Backend	LLaMA-3.1-8b-instant via Groq API
Environment Hashing	<code>hashlib</code> SHA-256 + <code>pip freeze</code>
Data Persistence	JSON-based episodic/semantic storage

B. Agent Architecture

Agent Architecture Four specialized agents divide the work. The Planner generates step-by-step execution plans using CoT prompting. The Coder produces code patches and pulls candidate strategies from SOMA when retrievals are available. The Executor runs code inside sandboxed Docker containers with no external network access. The Validator checks execution output against expected outcomes and reports pass/fail to the orchestrator. Agents communicate through a centralized message queue.

C. Data Flow

The user submits a bug report or coding task. The orchestrator parses the input, constructs the query tuple (T, E) , and hashes the current environment state into V . SOMA retrieves

the top- k candidate strategies scored by the interference-aware ranking in Equation (4).

If the top candidate already has a valid hash for the current environment, the Coder applies it directly. If not, and $\text{sim}_{\text{sem}} > \tau_{\text{probe}}$, Active Grounding probes the strategy in a sandbox first - a pass reuses the rule, while a failure hands off to the LLM for a fresh solution.

V. EXPERIMENTAL SETUP

We designed a multi-phase evaluation pipeline to test whether environment-conditioned memory actually reduces negative knowledge transfer. Unlike conventional LLM benchmarks that measure raw reasoning ability, our setup specifically isolates the memory architecture’s behavior through three complementary stages: (1) Controlled Drift Simulation, (2) Active Memory Grounding Benchmark, and (3) Micro-Real GitHub Evaluation.

All experiments ran on Ubuntu 22.04 LTS with an Intel Core i7-12700H (14 cores), 16 GB DDR5 RAM, Python 3.11.5, FAISS with 384-dimensional embeddings, and Docker 24.0. LLM inference went through the Groq API serving LLaMA-3.1-8b-instant.

A. Controlled Drift Benchmark

We simulated five API migrations common in Python ecosystems: the pandas 2.0 removal of `DataFrame.append()`, the OpenAI SDK switch to a client-based API pattern, the SQLAlchemy 2.0 ORM overhaul, Pydantic v2 configuration changes, and the `os.path` to `pathlib` migration.

Each experiment ran 50 sequential trials in two phases: Stationary Learning (Trials 1–25, old API) and Post-Drift Adaptation (Trials 26–50, new API). The agent received no notification of the change. We ran each of the five scenarios across 100 random seeds, yielding 12,500 total observations.

B. Configuration Sweep

We compared five memory configurations to isolate the contribution of each SOMA component:

- **LUMA Full SOMA:** Complete system with environment hashing, interference-aware retrieval, merging, and active grounding.
- **No Consolidation:** Environment-aware retrieval without the semantic consolidation step.
- **Naïve RAG:** Retrieves by cosine similarity separately, with no awareness of the live environment.
- **Vector Memory:** Logs environment metadata along with each of the strategy but ignores it at fetch time.
- **Episodic Only:** Saves raw execution traces as-is, with no abstraction or joining into reusable rules.

C. Robustness Sweep

We modified and changed the exploration rate from 0.50 to 0.90 to check that performance gains are attributable to the memory architecture, not to any of the particular parameter setting.

TABLE III
CONTROLLED DRIFT BENCHMARK RESULTS ($N=12,500$)

Config.	Pre Acc.	Post Acc.	NTR (%)	CFR (%)	Latency (trials)
LUMA Full SOMA	95.2	75.9	8.0	8.0	4
No Consol.	94.8	88.7	11.3	11.3	6
Naïve RAG	93.1	59.9	40.1	17.1	12
Vector Mem.	92.5	0.0	100.0	100.0	>25
Episodic Only	91.7	45.2	54.8	23.4	15

D. Evaluation Metrics

Three metrics structure the evaluation. NTR (Negative Transfer Rate) counts how often a post-drift trial recommends a wrong suggestion for the code change, incompatible with the new environment. CFR (Catastrophic Failure Rate) measures how often a stale strategy is actually executed. Adaptation Latency counts the number of trials required to stabilize after a glide event. CFR is used as the primary metric because a recommended but blocked incorrect fix is less harmful than one that is executed and fails.

E. Active Memory Grounding Evaluation

The grounding module was tested across seven setups: renamed modules (import path updated or changed), deprecated APIs (function removed), stable APIs, backward-compatible updates, partial alias behavior (some calls work while others do not), flaky environments (15% random failure rate), and multi-library shifts (two or more than two libraries changing at once). Each scenario ran 500 randomized trials.

F. Micro-Real GitHub Validation

Ten tasks were selected from documented GitHub issues involving real production system library upgrades in Pandas, SQLAlchemy, NumPy, and Matplotlib. Each task was executed three times through the full LUMA pipeline ($N = 30$).

VI. RESULTS AND DISCUSSION

CFR drives the analysis throughout this section. This metric reflects what typically occurs in operational environments: the agent applies a version-incompatible fix, the code executes, and the execution ultimately fails for the execution we have made.

A. Controlled Drift Benchmark

Table III summarizes results across all five configurations.

Baseline behavior. Vector Memory failed wholly post-drift, hitting 100% CFR. Logging environment metadata achieves no benefit when retrieval ignores. Naive RAG reached a 40.1% NTR, four in ten post-drift retrievals pointed to a diminished API. Also the baseline does not hold up once the environment shifts.

Accuracy vs. safety trade-off. No combination outperformed Full SOMA on raw post-drift accuracy (88.7% vs. 75.9%), but its CFR was 11.3% against SOMA’s 8.0%. The gap is expected from both of this. When SOMA detects

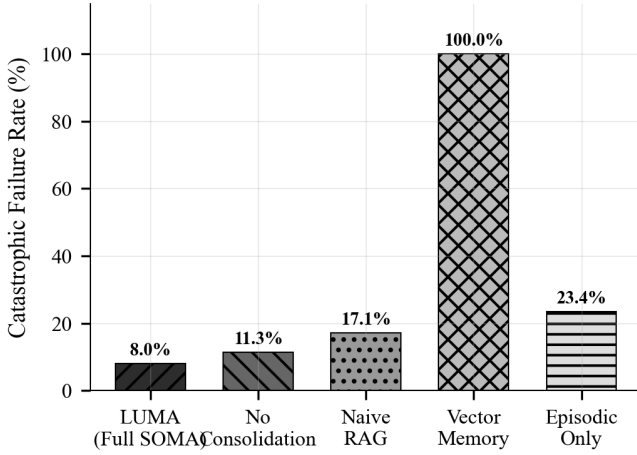


Fig. 2. CFR comparison across memory architectures under software drift.

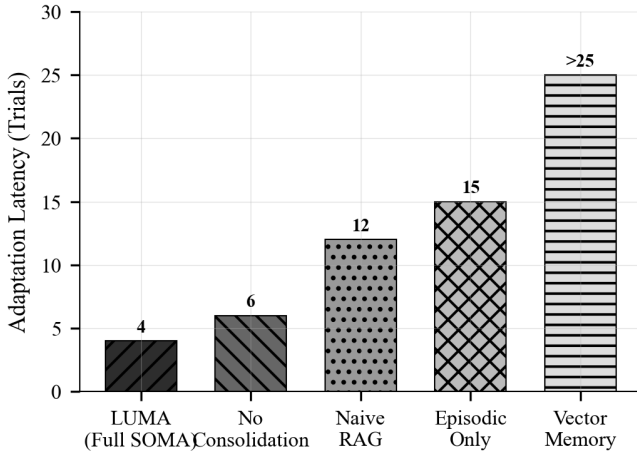


Fig. 3. Trials required to stabilize performance after a drift event.

environmental uncertainty, it re-evaluates or regenerates rather than pushing a potentially not correct fix through. On production code, trading a few correct points for fewer catastrophic failures is the correct thing.

Full SOMA stabilized within 4 trials after a drift event. Naïve RAG took 12. Vector Memory never stabilized within the 25-trial window.

Robustness across agent capabilities. Table IV shows LUMA maintaining a stable NTR near 8.0% across the complete analysis rate sweep. The result comes from the memory architecture, not from a lucky parameter setting. This answers RQ3.

B. Active Memory Grounding Evaluation

Table V reports grounding accuracy across seven scenarios.

On inevitable migration cases, the probe was almost correct. Under a flaky test environment with a 15% random failure rate, accuracy dropped down to almost 82.6%. Every error was a false negative: valid solution the probe incorrectly rejected. It

TABLE IV
ROBUSTNESS SWEEP: NTR (%) BY EXPLORATION RATE

Expl. Rate	LUMA	Naïve RAG	Vec. Mem.	Epis. Only
0.50	8.2	42.3	100.0	56.1
0.60	8.1	41.5	100.0	55.3
0.70	7.9	40.1	100.0	54.8
0.80	8.0	38.7	100.0	52.9
0.90	7.8	36.2	100.0	50.4

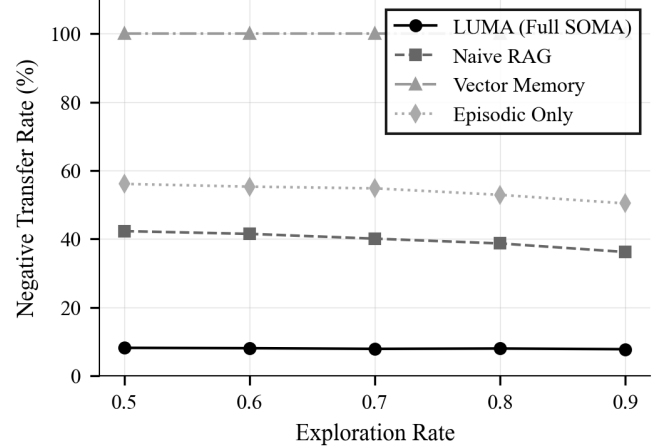


Fig. 4. Robustness of NTR across agent capability levels (exploration rate 0.50–0.90).

never approved a broken strategy. Zero false positives across all 500 trials. This answers RQ2.

Errors’ direction is important. Throwing away a potentially correct method wastes a retrieval. It corrupts the codebase to approve one that will break. The probe makes a mistake in the side of the former.

C. Micro-Real GitHub Validation

Pass@1 came in at 10.0% across 30 trials. The number is low, but the model driving it, LLaMA-3.1-8b-instant, has a constrained context window and was not designed for repository-scale refactoring. Failure modes were: 53% incomplete repository context, 30% incorrect API mapping, 17% syntax errors. None of them point the memory architecture.

The percentage of wrong modified files was 0.0%. The version-aware memory pipeline stopped the model from using outdated ideas against the codebase even when it was not able to fix a issue, thus the model safely failed.

D. Discussion

One reasonable question is if standard vector databases with manually curated version filters could do the same job. Possibly, but manual filtering requires someone to review and update compatibility rules every time a library releases a patch. At any meaningful scale, that maintenance burden becomes untenable. By creating environment hashes during validation

TABLE V
ACTIVE GROUNDING CLASSIFICATION RESULTS ($N=500$)

Scenario	Acc.	FPR	FNR
Stable API	100.0%	0.0%	0.0%
Deprecated API	100.0%	0.0%	0.0%
Renamed Module	100.0%	0.0%	0.0%
Backward-Compat. Update	100.0%	0.0%	0.0%
Partial Alias Behavior	95.2%	0.0%	4.8%
Flaky Test Env. (15%)	82.6%	0.0%	17.4%
Multi-Library Drift	91.4%	0.0%	8.6%

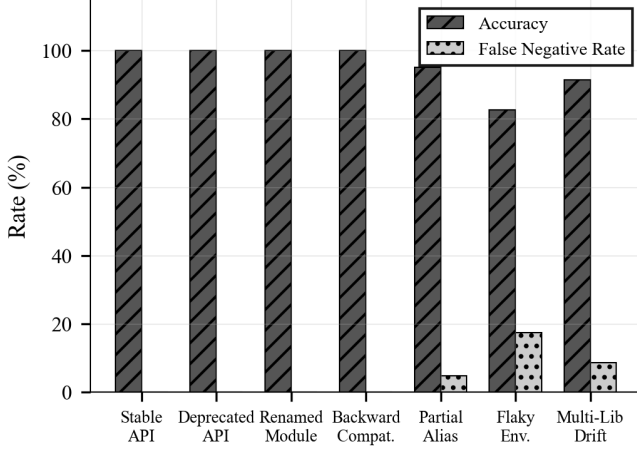


Fig. 5. Active grounding accuracy across the seven validation test case.

and using empirical probing to identify compatibility boundaries, LUMA tries to avoid it. There is no need for manual maintenance.

VII. PRACTICAL CONSIDERATIONS

A. Scalability

The episodic layer held roughly 3,200 entries, after 12,500 trials; the semantic layer had compressed these into 340 united rules. Retrieval latency stayed under 50 ms with FAISS indexing. Active Grounding adds 2–8 seconds per probe, manageable for batch workloads, may impact interactive performance.

B. Security

Each piece of code executes in separate Docker containers with strict disk rights and network access. All the request to external APIs are rate-limited and logged. In order to prevent adversarial or corrupted traces from accessing the memory pool, each candidate entry is verified before being written to SOMA.

C. Fault Tolerance

Retries with exponential backoff are started in answer to the short lived errors, while rules rules that fail probing are restricted for the specific environment that caused the failure. When no suitable strategy exists in memory, the system falls

back to fresh LLM generation. If SOMA is unavailable, the orchestrator proceed with stateless mode.

VIII. LIMITATIONS AND FUTURE WORK

A. Limitations

- **Sample Size Constraints:** Only 30 trials were used for real-world validation, which is insufficient to draw reliable statistical inferences on production-level performance.
- **Simulation Realism:** Instead than installing actual library versions, the controlled benchmark emulates API changes by switching expected outputs. The intricacy and messiness of actual dependency graphs are probably underestimated by this method.
- **Base Model Limitations:** Task success remains strictly bounded by the underlying language model’s capabilities. SOMA ensures safe retrieval but cannot compensate for a weak base LLM.
- **Latency Overhead:** The 2–8 second delay per Active Grounding probe introduces latency that may prove prohibitive for interactive or time-sensitive usecases.
- **Environment Hashing Scope:** The current hashing mechanism is limited to top-level dependencies, missing sub-dependency conflicts and system-level library variations unaccounted for.

B. Future Work

Most of the directions follow naturally from these limitations. Evaluation on SWE-bench Lite [6] would results hold in real-world situations. More detailed environment hashing could identify transitive dependencies. A United Collective SOMA would allow memory sharing across distributed agent teams. Evaluating with GPT-4-class models would clarify whether isolate architecture gains from raw model capability. Asynchronous grounding (running probes in the background while the agent handles other tasks) might reduce the latency concern. Extension to npm and Maven would test generalization beyond Python.

IX. CONCLUSION

LUMA is a distributed orchestration framework structured around SOMA, a version-aware memory system aimed at mitigating Software Drift. The main problem is that conventional retrieval systems treat stored knowledge as permanently valid. In software environments, that assumption fails, sometimes subtly, often repeatedly.

- 1) The results are direct. Environment-conditioned retrieval cut NTR from 40.1% (Naïve RAG) and 100% (Vector Memory) to 8.0%.
- 2) Active Grounding produced zero false positives across 500 trials.
- 3) SOMA’s reliability held across the full exploration rate sweep, ruling out configuration as the source of improvement.

Agents that operate over weeks and months will encounter environments that have shifted since their memory was written.

Semantic similarity has no way to detect that. Version-aware memory does.

X. THREATS TO VALIDITY

Internal Validity. The benchmark estimate API changes through output swapping instead of actual library installation. Real GitHub tasks partially address this, but complex transitive dependency interactions likely fall outside what the simulation captures.

External Validity. Testing covered five Python API migration changes. Generalization to other ecosystems (npm, Maven), other form of software drift we previously discussed (compiler flags, OS-level changes), or untested drift patterns also need investigation.

Construct Validity. NTR and CFR are proxy indicator. Production failures from deprecated code range from subtle data inconsistencies to complete system crashes, a severity spectrum our metrics do not fully distinguish.

ACKNOWLEDGMENT

The authors express their acknowledgment to the faculty of the School of Computer Engineering, KIIT University, for their guidance and for the computational resources that supported this work.

REFERENCES

- [1] S. Significant-Gravitas, "AutoGPT: An autonomous GPT-4 experiment," GitHub Repository, 2023.
- [2] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," *arXiv:2308.00352*, 2023.
- [3] X. Chen et al., "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv:2312.13010*, 2023.
- [4] L. Wang et al., "A survey on large language model-based autonomous agents," *Frontiers of Computer Science*, vol. 18, 2024.
- [5] C. E. Jimenez et al., "SWE-agent: Agent-computer interfaces enable automated software engineering," *arXiv:2405.15793*, 2024.
- [6] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" *arXiv:2310.06770*, 2023.
- [7] C. Packer et al., "MemGPT: Towards LLMs as operating systems," *arXiv:2310.08560*, 2023.
- [8] G. Wang et al., "Voyager: An open-ended embodied agent with large language models," *arXiv:2305.16291*, 2023.
- [9] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," in *Proc. NeurIPS*, 2023.
- [10] M. De Lange et al., "A continual learning survey: Defying forgetting in classification tasks," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2021.
- [11] J. Kirkpatrick et al., "Overcoming catastrophic forgetting in neural networks," *Proc. Natl. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017.
- [12] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, 2020.
- [13] G. I. Parisi et al., "Continual lifelong learning with neural networks: A review," *Neural Networks*, vol. 113, pp. 54–71, 2019.
- [14] C. Le Goues et al., "GenProg: A generic method for automatic software repair," *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, pp. 54–72, 2012.
- [15] M. Monperrus, "Automatic software repair: A bibliography," *ACM Comput. Surv.*, vol. 51, no. 1, 2018.
- [16] Z. Chen et al., "Neural transfer learning for repairing security vulnerabilities in C code," *IEEE Trans. Softw. Eng.*, vol. 49, no. 1, 2023.
- [17] J. Johnson et al., "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, 2021.
- [18] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019.
- [19] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," *Linux Journal*, 2014.
- [20] Y. Qin et al., "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proc. ICLR*, 2024.
- [21] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
- [22] T. Zhang et al., "CodeAgent: Enhancing code generation with tool-integrated agent systems," *arXiv:2401.07339*, 2024.
- [23] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv:2308.08155*, 2023.
- [24] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, 2017.